

CS358 Final Project Instructions

Part A

Overview

In this part your team will implement a parallel version of a prime-counting program using Java's **Fork/Join framework**. You are given two source files:

- **SerialPrimes.java** — a fully working serial implementation that counts all odd primes of the form $2i + 1$, where $1 \leq i < 999,999$.
- **ForkJoinPrimes.java** — a skeleton file that you must complete so that the same counting is done in parallel using Fork/Join.

Background: The Fork/Join Framework

Java's Fork/Join framework (introduced in Java 7) makes it easy to express divide-and-conquer parallelism. The key classes are:

- `RecursiveTask<V>` — a task that computes and returns a result of type `V`.
- `RecursiveAction` — a task that computes something but returns no result.
- `ForkJoinPool` — manages a pool of worker threads that steal work from each other.

The typical pattern for a `RecursiveTask` is:

```
class MyTask extends RecursiveTask<Integer> {
    @Override
    protected Integer compute() {
        if (rangeIsSmallEnough()) {
            return solveDirectly(); // base case
        }
        MyTask left = new MyTask(leftHalf);
        MyTask right = new MyTask(rightHalf);
        left.fork(); // schedule left asynchronously
```

```

        int rightResult = right.compute();

        int leftResult = left.join(); // wait for left

        return leftResult + rightResult;
    }
}

```

What is in Serial Version (SerialPrimes.java)

Read **SerialPrimes.java** carefully before writing any parallel code. The key points are:

- Odd numbers are represented as $2i + 1$, so the index i runs from 1 to 999,998.
- **isPrime(long num)** maps i back to $2i + 1$ and checks primality by trial division up to the square root.
- The serial loop simply calls isPrime for every index and accumulates a count.

Your Task: Complete ForkJoinPrimes.java

Step 1: Open **ForkJoinPrimes.java**. It already contains the same **main** method. Your job is to implement an isPrime method with a parallel Fork/Join solution.

Step 2: Update main. Remove the serial loop and the global pCount field. Instead, create a single PrimeTask covering the full range (1 to 999,999), you may invoke it via ForkJoinPool.commonPool().invoke(...), and print the returned count.

Step 3: Experiment with the threshold.

In the Fork/Join framework, the **threshold** is the cutoff point that decides when to stop splitting work into smaller sub-tasks and just solve the problem directly.

In this specific lab, the range of indices goes from 1 to 999,998. Each recursive call splits that range in half. The threshold answers the question: *"How small does a chunk need to be before we stop splitting and just loop through it?"*

For example, with a threshold of **500,000**:

- The root task sees 999,998 values → splits into two halves (~500,000 each)
- Each half is now $\leq 500,000$ → base case reached, count primes directly

- Result: **~2 tasks** running in parallel

With a threshold of **250,000**:

- Root splits into 2 → each half splits again into 2 more
- Result: **~4 tasks** running in parallel

The threshold matters because there's a tradeoff:

- **Too large** → few tasks, some CPU cores sit idle
- **Too small** → thousands of tiny tasks, and the overhead of creating/managing/joining them outweighs the benefit of parallelism

The goal of the experiment is for you to find the "sweet spot" where enough tasks are created to keep all CPU cores busy, without drowning in task management overhead.

Try the values listed in the table below, run each at least three times and record the average elapsed time.

Threshold Experiment

Use the following table to record your timing results. Fill in the *Approx. Tasks* column yourself after running.

Threshold	Approx. Tasks	Run 1 (ms)	Run 2 (ms)	Run 3 (ms)	Average (ms)	Speedup
500,000						
250,000						
100,000						
10,000						
1,000						
100						
Serial	1 (no threads)					

Discussion Questions

Answer these questions in a brief comment block at the top of your submitted *ForkJoinPrimes.java*:

Q1. How does the elapsed time change as the threshold decreases (more tasks created)? Is there a sweet spot, and why might that be?

Q2. What happens when the threshold is very small (e.g., 100)? Explain the overhead you observe.

Q3. How does your best parallel time compare to the serial time? What is the approximate speedup ($T_{\text{serial}}/T_{\text{parallel}}$) ratio?

Part-A Submission

Within your final project zipped folder, include your completed **ForkJoinPrimes.java** file (with discussion answers in the header comment) to the Final Project Exam link on BB. Make sure your code compiles and runs without errors before submitting. As good team programming practice, include the names of all team members in a comment block on top of your code file. Include the threshold table for discussion during your in-class presentation. The presentation file should also be included in the zipped folder.